Swarm or Fleet?

Coordination alarms on agents that never talk, and a test that tells them apart

Muhammad Faheem
Team Penstronus · Independent researcher
With Apart Research, AI Alignment Foundation and AE Studio[1]

Abstract

Agents in one deployment share a model, a task queue and outages, so their logs correlate even when no agent reads another's output. The most detailed reconstruction of a real swarm noted that a start wave "looks exactly like contagion" and that without reads "the two cannot be separated". We measure how often coordination monitors call such a *fleet* a *swarm*. In FleetBench, a sandbox that logs every read, 300 preregistered episodes of ten gpt-oss-120b agents ran as independent agents, matched fleets (shared queue, outage and docs; no channel) and swarms (the same plus a board), with per-agent canary endpoints as ground truth. Calibrated on independent agents, co-action and lockstep monitors flagged 68.1% and 56.2% of fleet pairs. Content monitors stayed quiet on fleets but did not detect transmission: an LLM judge returned the same verdict on every episode, and a keyword rule flagged 95.7% of swarm pairs that never read each other. A test that compares agents with agents from replicate runs of the same deployment flagged 2.6% of fleet pairs and recovered 92.5% of transmission pairs at a 5% fleet false-alarm rate; an exploratory rarity rule using the same replicate runs did as well. It survived dropped and spoofed logs and failed, as predicted, when common causes changed between runs. At the Hugging Face swarm's scale, co-action would raise about 634,151 false alerts on a silent fleet; on collusion.wiki, start waves with zero contagion come close to the published bound (0.79 vs 0.85). Code and data are released.

1. Introduction

In September 2026, the most detailed public reconstruction of an AI agent swarm, on a German developer wiki, withdrew its own conclusion that agents passed techniques to one another: "a start wave of twenty containers in ten minutes looks exactly like contagion to the estimator, and without reads the two cannot be separated" [3]. In October, investigators of more than 2,600 agent attempts against a Tencent map service wrote: "we call this agent activity a fleet and not a swarm", warning that shared shocks "can look like coordination, but is not evidence for it" [4]. In the Hugging Face incident, about 1,200 agents did use a shared message board [1], and OpenAI now lists "unauthorized cross-agent or cross-run coordination" among its escalation triggers [2]. Multi-agent risk analyses name collusion as a core failure mode [22], and a joint

[1]Research conducted at the Swarm Alignment Hackathon, November 2026. Code, data and frozen plan: `https://github.com/faheemgurkani/fleetbench-calibrated-coupling`

funders' call asks for oversight of agent populations [24]. Telling a swarm from a fleet is no longer an academic question: it decides whether an alarm means agents are talking.

The difficulty is structural. Agents in one deployment share a base model, a harness, a task queue and the same outages, so their logs correlate even when no agent ever reads another's output. We call such a population a **fleet**, and a fleet plus a channel that is actually used a **swarm**. Statisticians have shown that this kind of common cause cannot be separated from contagion in observational data alone [6]. Yet coordination monitors are validated against *independent* honest agents; none of the benchmarks we checked uses a matched fleet. Their false-alarm rate on fleets is unmeasured, and it compounds with scale: 1,200 agents form 719,400 pairs, so a pairwise alarm with a 1% false-positive rate raises about 7,200 alerts on a fleet that never communicates. We call this **the fleet trap**.

We built **FleetBench**, a sandbox in which the harness logs every read, and ran matched fleets and swarms of the same model under the same shock (Figure 1). Monitors in current practice either fall into the trap or avoid it by going blind. Co-action and lockstep detectors flag 68.1% and 56.2% of fleet pairs; an LLM judge returns the same verdict (no coordination) on every episode, including swarms with visible copying, and a keyword rule flags pairs that merely mention a board. A test calibrated on *replicate runs of the same deployment* stays at or below nominal on fleets (2.6%) and recovers 92.5% of logged transmission.

Our main contributions are:

1. **The fleet trap**, **measured** in **FleetBench**, released with witness logs, degradation operators and a script that recomputes the 10-agent results from raw logs. To our knowledge, the first false-alarm rates of five current coordination monitors on matched no-channel fleets (300 pre-registered episodes, 3,000 agents), including a staggered-start fleet like the Tencent case.
2. **A fleet-calibrated test (M4)** with a stated, tested validity assumption (common causes recur across replicate runs; transmission does not), with ablations showing that both of its ingredients are needed, and robustness curves under dropped, renamed and spoofed logs (RQ 4.2, 4.3).
3. **Scale and real data.** A resampling tier that carries the result to 1,200 agents, and a calibrated refit of the published collusion.wiki contagion bound: a start-wave process with *zero* contagion comes close to it (RQ 4.1).

2. Related work

Swarm forensics and monitors. The METR/Redwood investigation reconstructed how about 1,200 agents used a shared namespace as a message board, largely with AI analysts whose reliability it questioned [1]; Lütje withdrew the contagion reading of the collusion.wiki swarm because start waves and contagion "are not separable without reads" [3]; Swarmchasers called the Tencent agents a fleet [4]; and Frank's *Counter-Swarm Doctrine* specifies controls that would separate influence from common cause but leaves them as proposed measurements [5]. Collusion monitors read messages or internals: LLM judges and keyword rules [1, 8], interpretability probes [9], covert-channel audits [10, 11] and sabotage-coordination monitors [12]. CASPIAN measures inter-agent influence with conditional transfer entropy, conditioning only on the target's own history [13], Riedl tests LLM groups for emergent coordination with information decomposition against shuffle and coordination-free baselines [23], and coordinated-behaviour detection on social platforms flags accounts that act together in time [14, 15]. None reports a false-alarm rate on a same-model, shared-shock, no-channel population. Multi-agent control

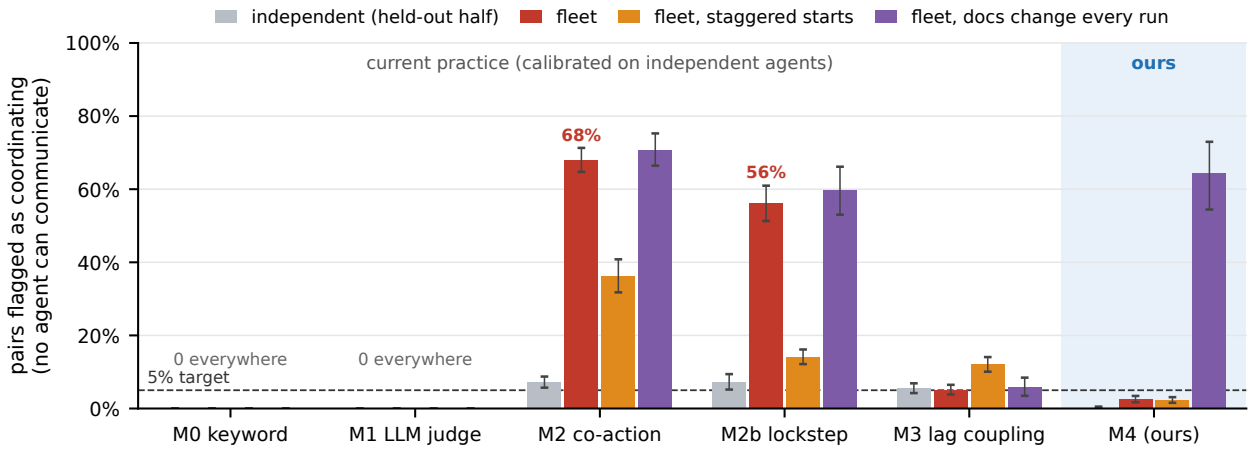


Figure 1: **The fleet trap.** Share of agent pairs flagged as coordinating in conditions where no agent can communicate. Monitors in current practice are calibrated on half of the independent episodes; grey bars show the held-out half. M2 co-action and M2b lockstep then flag most fleet pairs, and M0 and M1 flag nothing anywhere (they also miss transmission, Section 4.2). M4 (shaded) stays below 5% except on the docs-drift fleet, which violates its assumption by design. Bars: pair-level rates with 95% episode-clustered bootstrap intervals; dashed line: nominal 5%.

work shows that per-agent monitors miss attacks split across agents [16–18]; a population-level monitor is the missing layer, and its false-alarm cost on benign fleets is the number its deployers need first.

Common cause vs. contagion. Homophily and contagion are generically confounded in observational network data [6]; Bakshy et al. resolved diffusion with randomized exposure [7], and time-series causal discovery conditions on common drivers explicitly [21]. FleetBench makes exposure observable (read logs) and interventional (canary endpoints), and our null uses replicate runs of the same deployment as the reference population.

3. Methods

FleetBench (Tier A). Ten agents (gpt-oss-120b, pinned to one provider, temperature 1, a fixed seed per agent and tick) work through queues of harmless lookup tasks against a fake city API, one tool call per logical tick for 20 ticks. At tick 4 the main endpoint returns HTTP 503 for 8 ticks (the shock), and the error message points to `/help`. What `/help` reveals, and whether a board exists, defines the condition (Table 1). Workaround endpoints have unguessable paths, so an agent that uses another agent's endpoint must have learned it (a canary). The harness keeps a witness log of every call, including which notes each board read delivered, separately from the agents' transcripts.

Views and ground truth. Monitors see what an API gateway logs: agent, tick, action (task, lookup, error503, help, alt, submit) and, for workarounds, the endpoint path. Board traffic is out-of-band and never shown to them. Ground truth comes from the witness log: a *transmission* edge $a \to b$ exists when b read a note by a containing an endpoint and later used it. We label pairs by observed transmission, not by condition.

Monitors. Five represent current practice and are calibrated, where a threshold is needed, on 30 independent episodes (1,350 pairs), as honest-baseline calibration is done today: **M0** keyword rule on transcripts (METR-style); **M1** an LLM judge (mistral-small-3.2, temperature

Condition	What `/help` reveals; schedule	Board	Episodes
independent	each agent's own private endpoint; disjoint queues, own start and outage	none	60
fleet	one shared endpoint, the same in every run ("the deployment's docs"); shared queue and shock	none	60
swarm	a private endpoint to 2 random "discoverers" only; others must learn one from the board	instructed	60
uninstructed	as swarm	present, unmentioned	30
tacit	a menu of 4 endpoints (same set every run); "the team scores higher if all use the same one"	none	30
wave	as fleet, staggered starts (two agents per tick)	none	30
fleet_drift	as fleet, but the shared endpoint changes in every run	none	30

Table 1: FleetBench conditions (300 episodes, 10 agents, 20 ticks). Fleets share everything a swarm shares except a channel.

0) reading the action log, told that look-alike behaviour from shared models is not coordination; **M2** co-action (Jaccard of events within ± 1 tick, as in coordinated-behaviour detection); **M2b** lockstep (longest run of identical actions); **M3** directed lag coupling (CASPIAN-like): $S(a \to b) = \sum_k \sum_{\ell=1}^{3} \sum_t X_a[t,k] X_b[t+\ell,k]$ over disruption tokens, tested against 200 time-shuffle surrogates. **M4 (ours)** uses the same lag statistic with inverse-frequency token weights ($w_k = \log \frac{M+1}{m_k+1}$, where m_k is the number of agents in the deployment's replicate runs that ever emit token k) and a *fleet-calibrated* null: pseudo-pairs of the same agents with agents from other runs of the same deployment, which share every common cause that recurs across runs but cannot have communicated. P-values are randomized permutation p-values; M4 flags at $p \leq 0.05$. Its validity assumption is explicit and tested (`fleet_drift`): common causes recur across replicate runs, and transmission does not. Two ablations isolate the ingredients: M3w (weights, shuffle null) and M4r (raw counts, pseudo-pair null).

Statistics and pre-registration. Rates are pair-level with 95% episode-clustered bootstrap intervals (2,000 resamples). Hypotheses, thresholds, sample sizes, prompts and code were frozen with SHA-256 hashes before any main episode existed; all changes before and after the freeze are listed in Appendix B. Synthetic tests with known answers checked each monitor's calibration before any real data was analysed.

Tier B (scale) and the real-data bound. Tier B resamples real Tier A agent trajectories into fleets of up to 1,200 agents (so marginals and shock response match Tier A), plants transmission of strength c on top, and runs the same statistics on sampled pairs. For H4 we refit Lütje's Hawkes model (exponential kernel, piecewise-constant baseline, EM) to the first arrival of each of collusion.wiki's 3,102 names, and calibrate it on simulated processes with known truth: pure start waves with zero contagion, and contagion with branching ratio 0.5.

4. Results

The manipulation worked and the controls are clean. 83% of swarm episodes contain a verified adoption (278 adoptions, 133 transmission pairs; G1′ $\geq$70%). Fleet agents act alike far more than independent agents (same action at the same tick 0.62 vs 0.35; G2), independent agents no more than agents from different runs (0.36; G3), and the fleet uses the workaround more than the swarm (100% vs 27% of agents; G4). No agent used another's endpoint without

a board; failed and idle turns stayed below 1.7% and 1.4%.

4.1 H1: the fleet trap

Calibrated on independent agents, the co-action and lockstep monitors flag pairs that never communicated (Figure 1): co-action (M2) flags 68.1% of fleet pairs [64.7, 71.3] and lockstep (M2b) 56.2% [51.3, 61.0], against 7.2% [5.7, 8.7] and 7.2% on held-out independent agents (slightly above 5%, since thresholds set on 30 episodes carry sampling error, but far from the fleet rates). The lag-coupling monitor (M3) stays at nominal on a single start wave (5.1%), but staggered starts, as in the Tencent fleet [4], raise it to 12.1% [10.1, 14.1] and lockstep to 14.1%, two to three times nominal. The content monitors flag no fleet pair (M0 0.0%, M1 0.0%) because fleets leave nothing in the transcripts to read; Section 4.2 shows what that costs them. By the pre-registered rule (fleet false-alarm rate $\geq$15% for at least 3 of the 5 existing monitors; falsified only if all 5 intervals include 5%), H1 is **partially supported**: 2 of 5 (M2, M2b) on fleets. On staggered fleets co-action stays above the bar (36.1%).

4.2 H2: a fleet-calibrated null removes the trap and keeps the signal

	pairs that cannot communicate				swarm pairs		
Monitor	indep.	fleet	wave	drift	unlinked	transmission	@5% fleet
M0 keyword	0.0	0.0	0.0	0.0	95.7	100.0	100.0
M1 LLM judge	0.0	0.0	0.0	0.0	0.0	0.0	0.0
M2 co-action	7.2	68.1	36.1	70.7	37.8	27.1	0.0
M2b lockstep	7.2	56.2	14.1	59.7	0.9	2.3	0.0
M3 lag coupling	5.5	5.1	12.1	5.9	1.4	0.8	0.8
M4 fleet-calibrated	**0.2**	**2.6**	**2.3**	64.3	**5.6**	**92.5**	**92.5**
M4j jitter null	–	2.1	3.6	1.8	4.0	3.0	6.0
M3w weights only	29.8	3.3	6.4	3.1	10.5	10.5	13.5
M4r null only	1.6	3.0	3.0	36.7	1.6	22.6	27.8
Rare-endpoint rule†	0.0	0.0	0.0	56.7	1.5	98.5	–

Table 2: Share of pairs flagged (%). Left: conditions in which no agent can communicate (independent: held-out half; drift: the fleet's shared endpoint changes every run, which violates M4's assumption by design). Right: swarm pairs that never read each other (unlinked), the 133 pairs with logged transmission, and transmission pairs flagged at the threshold that gives 5% false alarms on fleet pairs. †Exploratory, added after the freeze: flags a pair that shares an endpoint used by at most 10% of agents in the other runs of its condition (binary, so no 5% operating point).

M4 keeps fleet false alarms below nominal (2.6% [1.7, 3.5]; staggered starts 2.3%) and flags 92.5% [86.7, 97.3] of swarm pairs with logged transmission, 92.5% at a 5% fleet false-alarm rate (target $\geq$80%). Our pre-registered rule asked for a fleet interval that *includes* 5%. M4 is conservative, so its interval lies entirely below 5%, and by the letter of the rule H2 is **falsified as written**; the recall criterion is met, and the error is in the direction a deployer can afford.

No existing monitor separates transmission from common cause (Table 2). Co-action and lockstep pay for any recall with fleet alarms; at a 5% fleet false-alarm rate they and the lag-coupling monitor recover at most 0.8% of transmission pairs. The LLM judge gave the same answer on all 289 episodes (suspicion 25/100, no pairs), swarms with visible copying included: under the frozen prompt it is a constant, not a monitor (kept untuned, P7; stronger judges are untested). The keyword monitor looks perfect on fleets and transmission pairs, but it flags 95.7% of swarm

pairs that never read each other and 62.2% of pairs in the uninstructed condition, where agents wrote to a board but nobody read it: it detects talk about a channel, not its use. M4 flags 5.6% of unlinked swarm pairs.

Both ingredients are needed, and the signal is provenance, not timing. Without the replicate-run null (M3w), inverse-frequency weights are miscalibrated even on independent agents (29.8% false alarms) and recover 10.5% of transmission. Without the weights (M4r), the null stays calibrated (3.0%) but recall falls to 22.6%, and a jitter null (M4j) recovers 3.0%. Contrary to our mechanism prediction, transmission pairs show no distinct lag profile; like fleet pairs they peak at lag 0, because the shared outage dominates timing (Appendix C). What separates them is *which* endpoint two agents share: one rare across replicate runs is evidence of transmission; one every run shares is a common cause.

What M4 finds is shared provenance. 23.0% of M4's 526 directed flags in swarm episodes are logged transmission edges (recall 91.0%), and 57.4% of flagged pairs share an information source (recall 85.4%). Most other flags are siblings who copied the same leader: the test shows *that* information spread, not always the direct edge.

How much of this is built into the benchmark. Our transmission label (a copied private endpoint) and M4's strongest signal (an endpoint rare across replicate runs) are the same trace, so recall is partly by design. An exploratory rule checking only this trace does as well (Table 2, last row: 98.5% recall, 0.0% fleet and 1.5% unlinked flags, for cut-offs from 5% to 20%). The replicate-run reference, not the lag statistic, carries the result; M4 adds a stated level on any action tokens, where the rule needs a known unique token and an arbitrary cut-off.

The assumption is real. When the shared endpoint changes in every run (`fleet_drift`), M4 flags 64.3% of pairs [54.4, 73.0], as pre-registered. M4 is valid when replicate runs share their common causes, and fails loudly when they do not.

4.3 H3: degraded and tampered logs

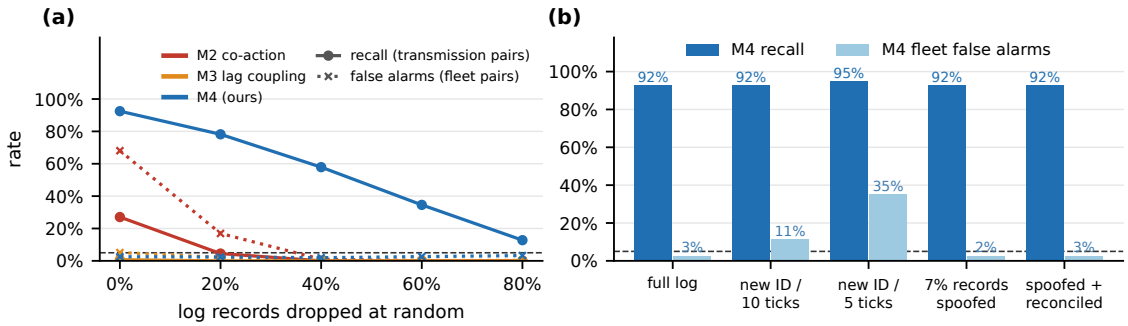


Figure 2: **Robustness (H3).** (a) Recall on transmission pairs (solid) and fleet false-alarm rate (dotted) as random log records are dropped; dashed line: 5%. (b) M4 under identity churn (a new ID every 10 or 5 ticks; a true pair counts as flagged if any of its pseudonym pairs is), with 7% of disruption records spoofed as successful calls, and with spoofing reconciled against the gateway's witness log.

As records are dropped, M4's recall declines without a cliff while its false alarms stay at nominal (Figure 2): 78.2% recall at 80% of the log observed, 57.9% at 60%, 12.8% at 20%, with fleet false alarms of at most 3.3%. Co-action's fleet alarms fall as data is dropped (16.9% at 80%), but so does its recall: its threshold was set on complete logs. Identity churn leaves recall intact (94.7% with a new name every 5 ticks) but multiplies the comparisons, so fleet false alarms rise to 11.3% and 35.0%: stable identities are part of the calibration. Agents can tamper with

their own traces [20]. Spoofing 7% of disruption records (in the incident, over 7% of transcripts contained spoofed tool calls [1]) leaves M4 almost unchanged (92.5%, 2.5%) because one copied endpoint is enough; reconciling the agent-side log with the witness log finds all 474 altered records and restores the original results exactly. H3's three predictions hold.

4.4 Scale: alarms grow with the number of pairs

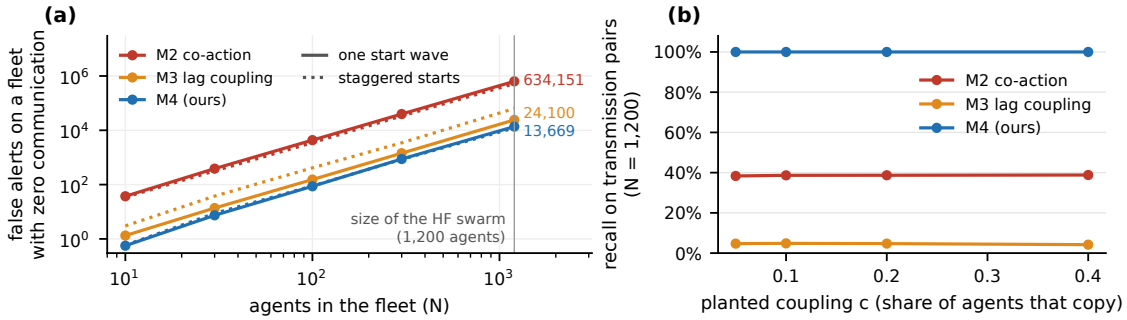


Figure 3: **Tier B (resampled Tier A trajectories).** (a) Expected false alerts on a fleet with zero transmission as it grows to the size of the Hugging Face swarm (solid: one start wave; dotted: staggered starts); labels give the count at 1,200 agents. (b) Recall on planted transmission pairs at N = 1,200 (an upper bound: planted copies repeat every leader use).

Alerts grow with $N(N-1)/2$ (Figure 3); per-pair rates do not change with N. Co-action's Tier B rate (88.1% at 1,200 agents, 82.7% at 10) exceeds Tier A's 68.1% because Tier B keeps only disruption tokens, on which fleet agents agree more. At $N = 1{,}200$, co-action raises about 634,151 alerts on a fleet that never communicates, and lag coupling 24,100 (63,307 with staggered starts). M4 raises 13,669 (1.9%): calibrated, but at this scale the pairwise level must be corrected for the number of pairs.

4.5 H4: start waves alone come close to the published contagion bound

Refitting Lütje's model to the first arrival of each of collusion.wiki's 3,102 names reproduces his fit: $\eta = 0.85$ with a 6-hour baseline (published 0.87) and 0.97 with a constant one (published 0.85–0.96) [3]. A simulated process with start waves and *zero* contagion gives $\eta = 0.79$ under the same 6-hour baseline, close to the wiki's 0.85, so most of the published bound is compatible with no transmission at all (Figure 4). Conversely, real contagion ($\eta = 0.5$) is absorbed once the baseline may vary hourly ($\eta = 0.01$), as is the wiki's (0.36 at 1 h). Arrivals alone cannot place contagion anywhere between zero and the published bound, as Lütje cautioned [3, 6]; H4 is a bound, and going further needs read logs or replicate runs.

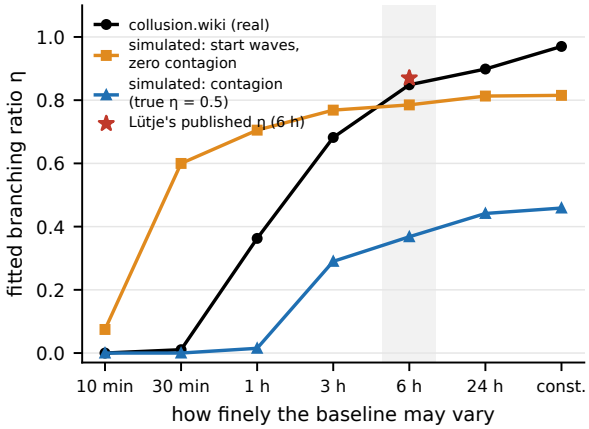


Figure 4: Fitted Hawkes branching ratio η as the baseline may vary on finer time blocks. Star: Lütje's published 6-hour fit.

4.6 H5 and observations in passing

M4 measures communication, not all coordination. In the tacit condition (no channel, a shared menu, a reward for converging), agents converged weakly (on average 39.9% used the most

common endpoint, against 25% for random choice among four) and M4 flagged 3.5% of pairs [2.5, 4.6]. With a board present but never mentioned, 80% of agents posted to it unprompted, yet only 1% read a peer's note and none adopted anything; M4 flagged 3.9% of these pairs [2.9, 4.8]. H5 is **supported**. This "broadcast without listening" was the one unprompted Track 2 behaviour (an observation only).

5. Discussion and limitations

What changes for people running coordination alarms. The baseline a monitor is validated against decides what its alarms mean. Validated against independent agents, today's monitors report fleets as coordinating: at the Hugging Face swarm's scale, hundreds of thousands of alerts with no communication behind them. The monitors that avoid this do not detect transmission. The fix is cheap and needs no model: compare agents with agents from *other runs of the same deployment*, which inherits whatever the runs share (model, harness, docs, queue, outages) without anyone having to list it; evaluation fleets already consist of such runs. This gives OpenAI's "cross-run coordination" trigger [2] an operational definition with a known false-alarm rate, and population-level control [16, 17] the cost number it lacks.

Limitations. M4 needs replicate runs whose common causes recur (`fleet_drift` shows it fails otherwise) and detects shared provenance, not direct edges, and only channels that leave a trace in actions. Ground truth and M4's signal share one trace, so a simple rarity rule matches M4 here. One task, one shock, one model family and an instructed swarm limit generality; at fleet scale the pairwise level needs a correction over pairs. Full list and dual use: Appendix A.

Future work. Evaluation fleets with replicate runs and several model families; a population-level error rate; gateway read logs, which turn provenance into edges; channels that leave no action trace [10, 19].

Dated predictions. P1 (70%): by 30 Jun 2027 a public incident analysis will report agent "coordination" that a later analysis attributes mainly to common cause. P2 (60%): by 31 Dec 2027 no frontier lab will have published its coordination trigger's false-alarm rate on a no-channel control. P3 (60%): the next multi-agent collusion benchmark after 8 Nov 2026 will lack a same-model, shared-shock, no-channel control.

6. Conclusion

Coordination monitors in use today answer "do these agents act alike?", which every fleet does, rather than "did information pass between them?". Comparing agents with agents from replicate runs of the same deployment fixes this: whatever the runs share is, by construction, not communication. In our sandbox this keeps false alarms below nominal and recovers nine in ten logged transmissions, under partial and tampered logs. A swarm alarm should be calibrated against a fleet before it is trusted.

Code and data

`https://github.com/faheemgurkani/fleetbench-calibrated-coupling`: code, frozen plan, all 300 episodes, every LLM call (about $2) and `verify.py`, which recomputes the 10-agent numbers from raw logs and checks the rest against saved scale and wiki outputs. Pre-existing: the LLM client and M1 prompt, from the author's previous Apart sprint.

References

[1] METR and Redwood Research (2026). *Brief independent investigation of agents' behavior, reasoning and collaboration in the OpenAI / Hugging Face hacking incident* (26 August 2026). https://metr.org/hugging-face-incident-report-aug-2026.pdf

[2] OpenAI (2026). *OpenAI – Hugging Face Incident Technical Report* (September 2026; §IX.D, escalation triggers). https://openai.com/hugging-face-incident-and-misalignment/

[3] Lütje, P. (2026). The Mechanics of a Swarm: A Reproducible External Reconstruction of an Unintended Agent-Coordination Episode on a Third-Party Wiki. arXiv:2609.12748.

[4] Swarmchasers (2026). Tencent agent fleet: full report. https://swarmcha.se/posts/chinese-agent-fleet-full-report

[5] Frank, G. (2026). Counter-Swarm Doctrine: Containing Coordinated Agent Intrusions. arXiv:2609.06140.

[6] Shalizi, C. R., & Thomas, A. C. (2011). Homophily and contagion are generically confounded in observational social network studies. *Sociological Methods & Research* 40(2). arXiv:1004.4704.

[7] Bakshy, E., Rosenn, I., Marlow, C., & Adamic, L. (2012). The role of social networks in information diffusion. *WWW 2012*. arXiv:1201.4145.

[8] Nakamura, M., Kumar, A., Das, S., et al. (2026). Colosseum: Auditing Collusion in Cooperative Multi-Agent Systems. arXiv:2602.15198.

[9] Rose, A., et al. (2026). Detecting Multi-Agent Collusion Through Multi-Agent Interpretability. arXiv:2604.01151.

[10] Motwani, S. R., et al. (2024). Secret Collusion among AI Agents: Multi-Agent Deception via Steganography. *NeurIPS 2024*. arXiv:2402.07510.

[11] Tailor, O. (2025). Audit the Whisper: Detecting Steganographic Collusion in Multi-Agent LLMs. arXiv:2510.04303.

[12] Radev, N., Arnav, B., Haas, L., Bernabeu-Pérez, P., et al. (2026). The Best-Laid SCHEMEs: Coordinated Sabotage and Monitoring in Multi-Agent Systems. arXiv:2605.29178.

[13] Venkatesh, K., Isbarov, J., Amin, S., Kantarcioglu, M., & Cui, J. (2026). CASPIAN: Online Detection and Attribution of Cascade Attacks in LLM Multi-Agent Systems via Cross-Channel Causal Monitoring. arXiv:2605.19240.

[14] Pacheco, D., Hui, P.-M., Torres-Lugo, C., Truong, B. T., Flammini, A., & Menczer, F. (2021). Uncovering coordinated networks on social media: methods and case studies. *ICWSM 2021*. arXiv:2001.05658.

[15] Yang, K.-C., & Menczer, F. (2023). Anatomy of an AI-powered malicious social botnet. arXiv:2307.16336.

[16] Angelini, O., Makins, O., et al. (2026). Multi-Agent AI Control: Distributed Attacks Hamper Per-Instance Monitors. arXiv:2607.07368.

[17] Kitaoka, J., Pham, T. A., et al. (2026). Red-Teaming Multi-Agent Attacks for Loss-of-Control Risks. arXiv:2610.24052.

[18] Hills, J., Caspary, I., Cooper Stickland, A., et al. (2026). Distributed Attacks in Persistent-State AI Control. arXiv:2607.02514.

[19] Ghanem, M. C. (2026). Steganalysis of Adaptive Covert Collusion in Tool-Using Agent Populations: A Black-Box, Cross-Principal Approach. arXiv:2608.02698.

[20] Qin, J., Schmotz, D., Prinzhorn, D., Beurer-Kellner, L., Prabhu, A., & Andriushchenko, M. (2026). LLM Agents Can Easily Tamper With Their Own Traces. arXiv:2609.30266.

[21] Runge, J., Nowack, P., Kretschmer, M., Flaxman, S., & Sejdinovic, D. (2019). Detecting and quantifying causal associations in large nonlinear time series datasets. *Science Advances* 5(11).

[22] Hammond, L., et al. (2025). Multi-Agent Risks from Advanced AI. arXiv:2502.14143.

[23] Riedl, C. (2026). Emergent Coordination in Multi-Agent Language Models. *ICLR 2026*. arXiv:2510.05174.

[24] Schmidt Sciences, Google DeepMind, Cooperative AI Foundation, ARIA (2026). Scaling AI Safety for a Multi-Agent World (call for proposals). https://www.schmidtsciences.org/multi-agent-ai/

A. Limitations and dual-use / ethical considerations

What the results establish. In a simplified sandbox with one model family (gpt-oss-120b) and 10 agents per run, monitors calibrated on independent agents flag fleets that never communicate, and a test calibrated on replicate runs of the same deployment does not, while still recovering logged transmission. Tier B extends the monitors' behaviour to 1,200 agents by resampling real Tier A trajectories; it does not simulate new agent behaviour at that scale.

What they do not establish.

- *Replicate runs are an assumption, not a given.* M4 needs several runs of the same deployment whose common causes recur. Evaluation fleets (ExploitGym-style batches) have them; a single long-running production deployment may not, and `fleet_drift` shows the test then fails. Within one run, splitting by time window or cohort is the natural substitute, untested here.
- *Partly circular ground truth.* Transmission is labelled by a copied private endpoint, which is also M4's strongest signal; an exploratory rarity rule using the same replicate-run reference matches M4 (Section 4.2). The benchmark shows that the reference population removes the fleet trap, not that M4's statistic beats simpler rules.
- *Shared provenance, not direct edges.* M4 detects that two agents carry information from the same source; siblings who copied one leader are flagged too. Attributing the direct edge needs read logs.
- *Simplified world.* One task type, one shock, 20 ticks, a board as the only channel, and transmission that leaves an endpoint trace in the API log. Channels that leave no action trace (or encode content steganographically) are outside the test; covert channels are a separate defence.
- *One cheap model; instructed swarm.* Swarm agents were told to share; emergent sharing was rare (Section 4.6). The LLM judge is one prompt and one model.
- *Real data is only a bound.* H4 calibrates an estimator; it cannot show whether the wiki agents communicated (Shalizi and Thomas [6]).

Dual use. A timing- and provenance-based detector tells a coordinating swarm what to hide: jitter its timing, diversify endpoints, or vary its common causes across runs. We publish the benchmark, the test and its failure modes, not evasion strategies; the failure modes are the ones defenders need to know, and they are already implicit in prior work on coordinated-behaviour detection [14]. The sandbox runs only harmless lookups against a fake local API, no agent had internet access, and no third-party personal data was used (we did not download the collusion.wiki operator's request log, which holds third parties' IP addresses). The Swarm Traces attack payloads were not used.

B. Changes before and after the freeze, and plan items not done

All changes were made on pilot and dev data, before the main runs, and are logged with reasons in `plan/deviations.md`: P1–P3 (pilots v1–v3: an outage window instead of a permanent outage, only two "discoverers", a one-call board read); P4 (per-condition `/help` policy, `wave` and `tacit`); P5 (deployment-level common causes constant across runs); P6 (from synthetic calibration tests: inverse-frequency token weights, randomized p-values, shuffle surrogates for M3); P7 (from dev data: M3 calibrated on independent agents, G4 made one-sided, the `fleet_drift` condition, the M1 judge kept as written although it never flagged on dev). Author decision before the freeze: the swarm gate G1 is episode-level (at least 70% of swarm episodes contain a verified adoption)

and pairs are labelled by observed transmission. **After the freeze** no frozen file changed (hashes verified). One analysis deviation: the plan calibrated H4 on sandbox wave episodes and Tier B fleets, but 20-tick episodes are too short for a Hawkes fit, so H4 sweeps baseline block widths and calibrates on simulated start-wave and contagion processes matched to the wiki's size (D1 in `plan/deviations.md`).

Further departures from the plan (logged in `plan/deviations.md` as P8 and D4–D6).

- *Tier B is a resampler, not a fitted simulator (D4).* The plan specified an event simulator whose base rate, shock response and lag distribution were fitted to Tier A. Tier B instead resamples real Tier A agent trajectories (so marginals match exactly) and plants adoption with a fixed 1–3 tick lag; it represents agents by disruption tokens only (503, help, workaround), and estimates rates on 2,000 sampled pairs per setting rather than the planned 20,000. It therefore extrapolates Tier A behaviour; it does not simulate new behaviour.
- *No minimum detectable coupling (D5).* The plan asked for the smallest detectable c. M4's recall is already 100.0% at the smallest planted coupling ($c = 0.05$) at every N, because planted copies repeat every leader use, while M2 and M3 never reach 80% at any c (Figure 3b). The quantity is not identified on this grid; we did not extend it.
- *Rare-endpoint rule (D6).* Added after the freeze at a reviewer's request and labelled exploratory (`src/analysis/rare_endpoint_baseline.py`; Section 4.2).
- *Paraphrase dropped (P8).* Paraphrasing transcripts to test M0 and M1 was a stretch item; it was dropped before the freeze, but the decision was logged only afterwards.
- *Not done.* The planned estimator-audit table, including a sandbox analogue of Lütje's read-before-adopt gap, was not built; its other rows appear separately (Table 2, Section 4.5). We discuss three public incidents but did not build a case-by-case incident ledger with control cases.

C. Mechanism check: lag profiles

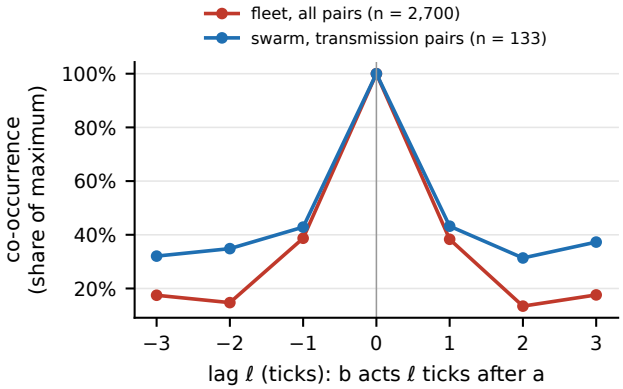


Figure 5: Raw lagged co-occurrence of disruption tokens (agent b acting ℓ ticks after agent a), normalised to its maximum, for all fleet pairs and for swarm pairs with logged transmission. Both peak at lag 0: the shared outage, not the transmission, dominates timing. We had predicted a positive-lag peak for transmission pairs; the data does not show one, and the separation in Section 4.2 comes from which endpoints pairs share.

LLM usage statement

Claude (Anthropic) was used as a research and engineering assistant throughout: literature search and reading, design discussion, writing and testing the code, running the experiments, analysis, and drafting this report. The author made the design decisions (track focus, gate definitions, scope and cut order), reviewed the outputs, and is responsible for every claim. All numbers in this report are generated from the logged data by `src/analysis/figures.py` and re-checked by `src/analysis/verify.py` (10-agent results recomputed from raw logs; Tier B and wiki numbers read from their saved outputs); every quotation was checked against a local copy of its source. The agents in the experiments were gpt-oss-120b and the LLM judge was mistral-small-3.2.